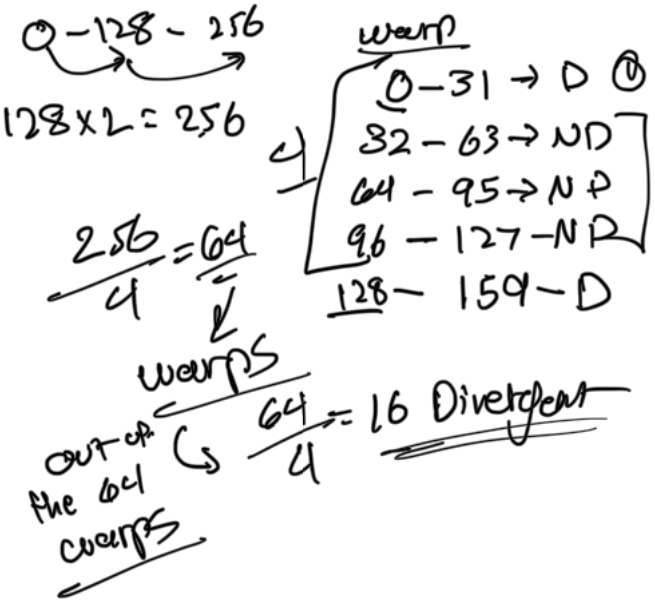
Midterm Review

UNIVERSITY OF CALIFORNIA, RIVERSIDE

[10 points] For the following basic reduction kernel code fragment, if the block size is 2048 and warp size is 32, how many warps in a block will have divergence during the iteration where stride is equal to 1? Stride equal to 32? Stride equal to 128?

```
for (unsigned int stride = 1; stride <= blockDim.x; stride *= 2)
{
   __syncthreads();
   if (threadIdx.x % stride == 0) {
     partialSum[2*threadIdx.x]+= partialSum[2*threadIdx.x + stride];
   }
}
```



1: 0 32: 64 128: 16

[10 points] For the following basic reduction kernel code fragment, if the block size is 2048 and warp size is 32, how many warps in a block will have divergence during the iteration where stride is equal to 1? Stride equal to 32? Stride equal to 128?

```
for (unsigned int stride = 1; stride <= blockDim.x; stride *= 2)
{
    __syncthreads();
    if (threadIdx.x % stride == 0) {
      partialSum[2*threadIdx.x]+= partialSum[2*threadIdx.x + stride];
    }
}
```

1: _____0_____ 32: _____64_____ 128: _____16_____

[20 points] In a parallel Reduction implementation, each thread loads two input elements from global memory to shared memory. The input elements are stored in:

__shared__ float partialSum[2*blockDim.x]; → each thread holds 2 elements

For simplicity, you do not need to do boundary condition checking. Answer the following.
(hint: start with the thread indexing first.)

Complete the code for a thread to load two adjacent input elements (i.e. thread 0 loads [0], [1], thread 1 loads [2],[3], etc.):

each thread holds 2 elements

unsigned int start = 2*blockIdx.x*blockDim.x;

unsigned int t = 2* threadIdx.x

partialSum[t] = input[start + (t)]

partialSum[t+1] = input[start + (t+1)]

[20 points] In a parallel Reduction implementation, each thread loads two input elements from global memory to shared memory. The input elements are stored in:
__shared__ float partialSum[2*blockDim.x];

For simplicity, you do not need to do boundary condition checking. Answer the following.
(hint: start with the thread indexing first.)

Complete the code for a thread to load two adjacent input elements (i.e. thread 0 loads [0], [1], thread 1 loads [2],[3], etc.):

unsigned int start = 2*blockIdx.x*blockDim.x;

unsigned int t = 2*threadIdx.x

partialSum[t] = input[start + t]

partialSum[t+1] = input[start + t + 1]

[20 points] In a parallel Reduction implementation, each thread loads two input elements from global memory to shared memory. The input elements are stored in:
__shared__ float partialSum[2*blockDim.x];

For simplicity, you do not need to do boundary condition checking. Answer the following.
(hint: start with the thread indexing first.)

Complete the code for a thread to load two input elements with a stride of block dimension:

unsigned int start = 2*blockIdx.x*blockDim.x;

unsigned int t = _____

partialSum[t] = input[start + _____]
partialSum[blockDim.x+t] = input[start + _____]

[20 points] In a parallel Reduction implementation, each thread loads two input elements from global memory to shared memory. The input elements are stored in:
__shared__ float partialSum[2*blockDim.x];

For simplicity, you do not need to do boundary condition checking. Answer the following.
(hint: start with the thread indexing first.)

Complete the code for a thread to load two input elements with a stride of block dimension:

unsigned int start = 2*blockIdx.x*blockDim.x;

unsigned int t = threadIdx.x

partialSum[t] = input[start + t]
partialSum[blockDim.x+t] = input[start + blockDim.x + t]

[10 points] You need to write a kernel that operates on an image of size 400x900. You would like to allocate one thread to each pixel. You would like the thread blocks to be square and to use the maximum number of threads per block possible on the device (assume 1536 thread limit and 8 thread block limit).

a) [4 points] What would you select as the grid and block dimensions?

b) [6 points] Assuming next that we use blocks of size 16x16, how many warps would experience thread divergence?

[10 points] You need to write a kernel that operates on an image of size 400x900. You would like to allocate one thread to each pixel. You would like the thread blocks to be square and to use the maximum number of threads per block possible on the device (assume 1536 thread limit and 8 thread block limit).

a) [4 points] What would you select as the grid and block dimensions?

16x16 would give us 6 blocks of 256 threads each, maximizing the number of threads in the SM

b) [6 points] Assuming next that we use blocks of size 16x16, how many warps would experience thread divergence?

The right-most block overhangs. Each warp (32-threads) spans 2 rows (2x16). Therefore, there is 200 warps with divergence.

[20 points] For the below Vector Add kernel answer the following questions. Assume a vector size of *n*, block size of 256 threads and (*n*-1)/256+1 thread blocks.

```
__global__ void VecAdd(int n,const float *A,const float *B,float* C) {
        for (unsigned int i = 0; i<n; i++){
                C[i] = A[i] + B[i];
        }
}
```

a) [4 points] Does the kernel produce the desired results?

b) [6 points] How many additions are performed in this VecAdd compared to the VecAdd you implemented for assignment 1? (Do NOT count i++ in the loop statement as an addition.)

[20 points] For the below Vector Add kernel answer the following questions. Assume a vector size of *n*, block size of 256 threads and (*n*-1)/256+1 thread blocks.

```
__global__ void VecAdd(int n,const float *A,const float *B,float* C) {
        for (unsigned int i = 0; i<n; i++){
                C[i] = A[i] + B[i];
        }
}
```

a) [4 points] Does the kernel produce the desired results?

Yes

b) [6 points] How many additions are performed in this VecAdd compared to the VecAdd you implemented for assignment 1? (Do NOT count i++ in the loop statement as an addition.)

Let nBlocks = ((n-1)/256+1)

This VecAdd: 256*nBlocks*n additions

Lab 1 VecAdd: n additions

[20 points] For the below Vector Add kernel answer the following questions. Assume a vector size of *n*, block size of 256 threads and (*n*-1)/256+1 thread blocks.

```
__global__ void VecAdd(int n,const float *A,const float *B,float* C) {
        for (unsigned int i = 0; i<n; i++){
                C[i] = A[i] + B[i];
        }
}
```

c) [6 points] How many total loads to memory are performed in this VecAdd compared to the VecAdd you implemented in assignment 1?

d) [4 points] Would this parallel VecAdd kernel run faster, or a serial implementation of Vector Add?

[20 points] For the below Vector Add kernel answer the following questions. Assume a vector size of *n*, block size of 256 threads and (*n*-1)/256+1 thread blocks.

```
__global__ void VecAdd(int n,const float *A,const float *B,float* C) {
        for (unsigned int i = 0; i<n; i++){
                C[i] = A[i] + B[i];
        }
}
```

c) [6 points] How many total loads to memory are performed in this VecAdd compared to the VecAdd you implemented in assignment 1?

Let nBlocks = ((n-1)/256+1)

This VecAdd: 2 * 256 * nBlocks * n loads

Lab 1 VecAdd: 2 * n loads

d) [4 points] Would this parallel VecAdd kernel run faster, or a serial implementation of Vector Add?

Most likely the serial implementation would be faster as both code will run in O(n) time, but GPU has overhead of memory allocation/data transfer.

[20 points] For the following, explain how it could harm performance and possible ways the program can be modified to reduce this effect. Please be specific.

a) [10 points] The application needs to access global memory to get one value for every operation.

How does this harm performance?

Technique/change that could reduce this effect:

[20 points] For the following, explain how it could harm performance and possible ways the program can be modified to reduce this effect. Please be specific.

a) [10 points] The application needs to access global memory to get one value for every operation.

How does this harm performance?

Memory access is slow. Can cause stalls in computation and bottleneck the memory.

Technique/change that could reduce this effect:

Using shared memory to cache data and maximize reuse.

[10 points] Control divergence: (aka Warp divergence)

How this harms performance:

Technique/change that could reduce this effect:

[10 points] Control divergence: (aka Warp divergence)

How this harms performance:

When threads in a warp execute different code paths (for example, if-else statements), all threads must execute the same instruction (all threads in a warp shares the same program counter). Therefore, threads that do not need to execute certain instructions will be idle, causing under-utilization of the hardware.

Technique/change that could reduce this effect:

You can change thread indexing, like in reduction, to minimize divergence.

a) DMA (Direct Memory Access) hardware transfers data between

virtual addresses
memory mapped addresses
mythical addresses
physical addresses
imaginary addresses

b) cudaMalloc allocates memory in:

host memory
pinned memory
device memory
shared memory
persistent memory

c) cudaHostAlloc allocates memory in:

shared memory
pinned memory
device memory
global memory
persistent memory

d) If your CUDA application has a single stream, can you concurrently copy data and execute a kernel?

Yes
No

e) Each CUDA stream is a _____ of operations

Event
Command
Heap
Queue
Stack

a) DMA (Direct Memory Access) hardware transfers data between

virtual addresses
memory mapped addresses
mythical addresses
physical addresses
imaginary addresses

b) cudaMalloc allocates memory in:

host memory
pinned memory
device memory
shared memory
persistent memory

c) cudaHostAlloc allocates memory in:

shared memory
pinned memory
device memory
global memory
persistent memory

d) If your CUDA application has a single stream, can you concurrently copy data and execute a kernel?

Yes
No

e) Each CUDA stream is a _____ of operations

Event
Command
Heap
Queue
Stack

f) Commands (aka Events) in CUDA streams can be processed out-of-order.

True

False

g) Which of the following is false?

1. Events in CUDA streams are processed in FIFO order
2. The OS can accidentally swap out a page that is being transferred by DMA
3. Kernel launches are CUDA events
4. Copies are CUDA events
5. All CUDA API calls are CUDA events

h) For the following questions answer whether it is (a) true or (b) false.

1. Virtual memory addresses are translated to physical memory addresses using page tables.

2. All warps in a thread block must execute the same instruction simultaneously.

3. Warps can finish executing at different times.

4. In the GPU, a scheduler exists to pick warps to issue for execution.

5. PTX is an intermediate representation for GPU code.

6. GPUs can boot an Operating System.

f) Commands (aka Events) in CUDA streams can be processed out-of-order.

True

False

g) Which of the following is false?

1. Events in CUDA streams are processed in FIFO order
2. The OS can accidentally swap out a page that is being transferred by DMA
3. Kernel launches are CUDA events
4. Copies are CUDA events
5. All CUDA API calls are CUDA events

h) For the following questions answer whether it is (a) true or (b) false.

1. Virtual memory addresses are translated to physical memory addresses using page tables. T

2. All warps in a thread block must execute the same instruction simultaneously. F

3. Warps can finish executing at different times. T

4. In the GPU, a scheduler exists to pick warps to issue for execution. T

5. PTX is an intermediate representation for GPU code. T

6. GPUs can boot an Operating System. F

Multiple Choice. Please completely fill in the circleofor your answer. [12 points total]

6. [3 points] If a CUDA device's SM (streaming multiprocessor) can take up to 1,536 threads and up to 8 thread blocks. Which of the following 1D block configuration would result in the greatest number of threads in each SM?

o 1,024 threads per block
o 64 threads per block
o 256 threads per block
o 128 threads per block

7. [3 points] Which of the following condition checks will cause warp divergence?

o If (threadIdx.x > 4)
o If (blockDim.x > 4)
o If (blockIdx.x > 4)
o None of the above

Multiple Choice. Please completely fill in the circleofor your answer. [12 points total]

6. [3 points] If a CUDA device's SM (streaming multiprocessor) can take up to 1,536 threads and up to 8 thread blocks. Which of the following 1D block configuration would result in the greatest number of threads in each SM?

o 1,024 threads per block
o 64 threads per block
o 256 threads per block
o 128 threads per block

7. [3 points] Which of the following condition checks will cause warp divergence?

o If (threadIdx.x > 4)
o If (blockDim.x > 4)
o If (blockIdx.x > 4)
o None of the above